

Chapter 3: System Architecture & Methodology

Tarmica Chiwara

Table of Contents

1	Chapter 3: System Architecture & Methodology	2
1.1	3.1 Introduction	2
1.1.1	3.1.1 Problem Statement	3
1.1.2	3.1.2 Target Users	3
1.1.3	3.1.3 System Goals	3
1.1.4	3.1.4 Scope & Constraints	4
1.2	3.2 Development Methodology.....	4
1.2.1	3.2.1 Approach: Evolutionary Prototyping.....	4
1.2.2	3.2.2 Development Phases	5
1.2.3	3.2.3 Tools & Environment	5
1.3	3.3 Hardware Architecture	6
1.3.1	3.3.1 Component Specification	6
1.3.2	3.3.2 Hardware Integration Diagram	7
1.3.3	3.3.3 CAN Bus Physical Layer.....	7
1.3.4	3.3.4 Power Management	8
1.3.5	3.3.5 Physical Installation Considerations	8
1.4	3.4 Software Architecture	8
1.4.1	3.4.1 Design Philosophy.....	8
1.4.2	3.4.2 Language Choice: Rust.....	9
1.4.3	3.4.3 Software Stack.....	9
1.4.4	3.4.4 Module Architecture.....	10
1.4.5	3.4.5 Threading Model	11
1.4.6	3.4.6 Memory Layout	12
1.4.7	3.4.7 Build Configuration	12
1.4.8	3.4.8 Dependencies	12
1.5	3.5 Data Acquisition Layer	13

1.5.1	3.5.1 OBD-II Protocol Overview	13
1.5.2	3.5.2 CAN Addressing Scheme	13
1.5.3	3.5.3 OBD-II Frame Format	14
1.5.4	3.5.4 ISO-TP Transport Layer	15
1.5.5	3.5.5 Parameter Selection (PIDs)	15
1.5.6	3.5.6 Discovery Protocol	16
1.6	3.6 Data Processing Pipeline	16
1.6.1	3.6.1 Main Loop Flow	16
1.6.2	3.6.2 Error Recovery	17
1.6.3	3.6.3 Data Logging Strategy	18
1.6.4	3.6.4 Timing Constraints	18
1.7	3.7 Testing & Validation Strategy	19
1.7.1	3.7.1 Virtual CAN Testing	19
1.7.2	3.7.2 Unit Tests	19
1.7.3	3.7.3 Integration Testing	20
1.7.4	3.7.4 Real Vehicle Testing (Planned)	20
1.7.5	3.7.5 Validation Metrics	20
1.8	3.8 Platform Portability	21
1.8.1	3.8.1 Cross-Platform Compilation Strategy	21
1.8.2	3.8.2 Deployment Procedure	21
1.9	3.9 Limitations & Future Considerations	21
1.9.1	3.9.1 Current Limitations	21
1.9.2	3.9.2 Security Considerations	22
1.9.3	3.9.3 Planned Enhancements	22
1.10	3.10 Conclusion	23

1 Chapter 3: System Architecture & Methodology

1.1 3.1 Introduction

This chapter presents the system architecture, design methodology, and technical implementation of KnightRider, a field-grade ECU diagnostic computer built for the Raspberry Pi platform. The chapter details the hardware composition, software

architecture, data acquisition strategy, and the protocols governing communication between the diagnostic tool and vehicle Electronic Control Units (ECUs).

1.1.1 3.1.1 Problem Statement

In Zimbabwe and much of the developing world, automotive diagnostics remains inaccessible to most people. Commercially available OBD-II scan tools carry prohibitive import costs, locking out independent mechanics, small workshops, and individual vehicle owners from understanding their own vehicle's health. Diagnostics persists as a separate, costly layer in the automotive repair industry, one that is fundamentally an information problem.

KnightRider addresses this gap by providing an open, affordable, and self-contained diagnostic platform using commodity hardware. The system is designed to operate fully offline, requiring no phone, cloud connectivity, or external dependencies, making it suitable for deployment in environments with unreliable or absent internet infrastructure.

1.1.2 3.1.2 Target Users

The system is designed for three primary user categories:

1. **Independent Mechanics and Workshops** - Small-scale repair shops in Zimbabwe and similar markets that currently lack access to professional diagnostic tools.
2. **Fleet Owners** - Operators of commercial vehicle fleets seeking to monitor vehicle health without recurring subscription costs.
3. **Technically Proficient Vehicle Owners** - Individuals who wish to understand their vehicle's operational state beyond the dashboard warning lights.

1.1.3 3.1.3 System Goals

The KnightRider system has been designed to achieve the following objectives:

- **Real-time vehicle diagnostics** - Read live sensor data from vehicle ECUs at polling rates sufficient for meaningful analysis (10 Hz for critical parameters).
- **Protocol compliance** - Implement ISO 15765-4 (CAN-based OBD-II), ISO 15765-2 (ISO-TP transport), and ISO 15031 / SAE J1979 (diagnostic services) to ensure compatibility with the majority of post-2008 vehicles.
- **Offline-first operation** - Function as a fully self-contained unit requiring zero network connectivity for core diagnostics.
- **Reliability in harsh environments** - Operate dependably in the vibration, temperature, and electrical noise conditions found in automotive environments.
- **Brand-agnostic compatibility** - Support the standardised OBD-II protocol layer common to 85%+ of vehicles manufactured since 2008, while providing extensibility for manufacturer-specific parameters.

1.1.4 3.1.4 Scope & Constraints

The current version (v1) of KnightRider operates within the following boundaries:

Constraint	Description
Supported Protocols	ISO 15765-4 (CAN-based OBD-II) only. Non-CAN protocols (K-Line, J1850 VPW/PWM) are explicitly out of scope.
CAN Bus Speeds	500 kbps (default, per ISO 15765-4) with 250 kbps fallback for older European vehicles.
OBD-II Services	Mode 01 (Current Data) only. Freeze frame, DTC reading, and manufacturer-specific modes are deferred to future versions.
Vehicle Compatibility	Post-2008 vehicles employing CAN-based OBD-II. Older vehicles using legacy protocols are not supported.
Display	Console and CSV output. Graphical UI is reserved for future development.
Network Connectivity	Not required. All processing occurs locally on the Pi.

1.2 3.2 Development Methodology

1.2.1 3.2.1 Approach: Evolutionary Prototyping

The project follows an **evolutionary prototyping** methodology, chosen for its suitability to hardware-software co-development where requirements are clarified through iterative experimentation with real vehicle data. Unlike throwaway prototyping, each prototype iteration in KnightRider becomes the foundation for the next. The v1 CAN reader module, for instance, is designed for production use from its first implementation.

This methodology was selected over alternatives for the following reasons:

- **Waterfall** was rejected because the project's requirements depend on empirical observation of real CAN bus traffic across different vehicle makes, which cannot be fully specified in advance.
- **Agile/Scrum** was considered but deemed overly process-heavy for a small team working on a tightly coupled hardware-software system.
- **Evolutionary Prototyping** allows the team to build a working system incrementally, validate each subsystem against real vehicle data, and refine requirements based on observed behaviour.

1.2.2 3.2.2 Development Phases

The project is structured into the following phases:

Phase	Description	Status
Phase 1: Scaffolding	Core software architecture in Rust. CAN bus module foundations, state machine, logging pipeline.	✓ Complete
Phase 2: Hardware Sourcing	Procurement of Raspberry Pi 5, Waveshare CAN HAT, OBD-II connectors, and power supply components.	✓ Complete
Phase 3: Upstream Setup	Software-side preparation for real data acquisition. Virtual CAN testing. Protocol implementation.	In Progress
Phase 4: Lab Testing	Experimental phase with live vehicles at partner workshop (Toyo-Tech). CAN data recording and protocol validation.	Planned
Phase 5: Hardware V1	First field-ready units with live CAN data analysis and integrated display.	Planned

1.2.3 3.2.3 Tools & Environment

Tool	Purpose
Rust 2021 Edition	Primary implementation language, chosen for memory safety without garbage collection overhead. This is critical for embedded automotive systems.
Cargo	Rust's package manager and build system. Manages dependencies and cross-compilation.
Linux SocketCAN	Kernel-level CAN bus subsystem providing a socket-based API.
Virtual CAN (vcan)	Linux kernel module enabling software CAN bus simulation for development without physical hardware.
can-utils	Linux command-line tools (cansend, candump) for CAN bus debugging and ECU response simulation.
Git / GitHub	Version control and project hosting.
Raspberry Pi OS Lite	Operating system for the target deployment platform.

1.3 3.3 Hardware Architecture

1.3.1 3.3.1 Component Specification

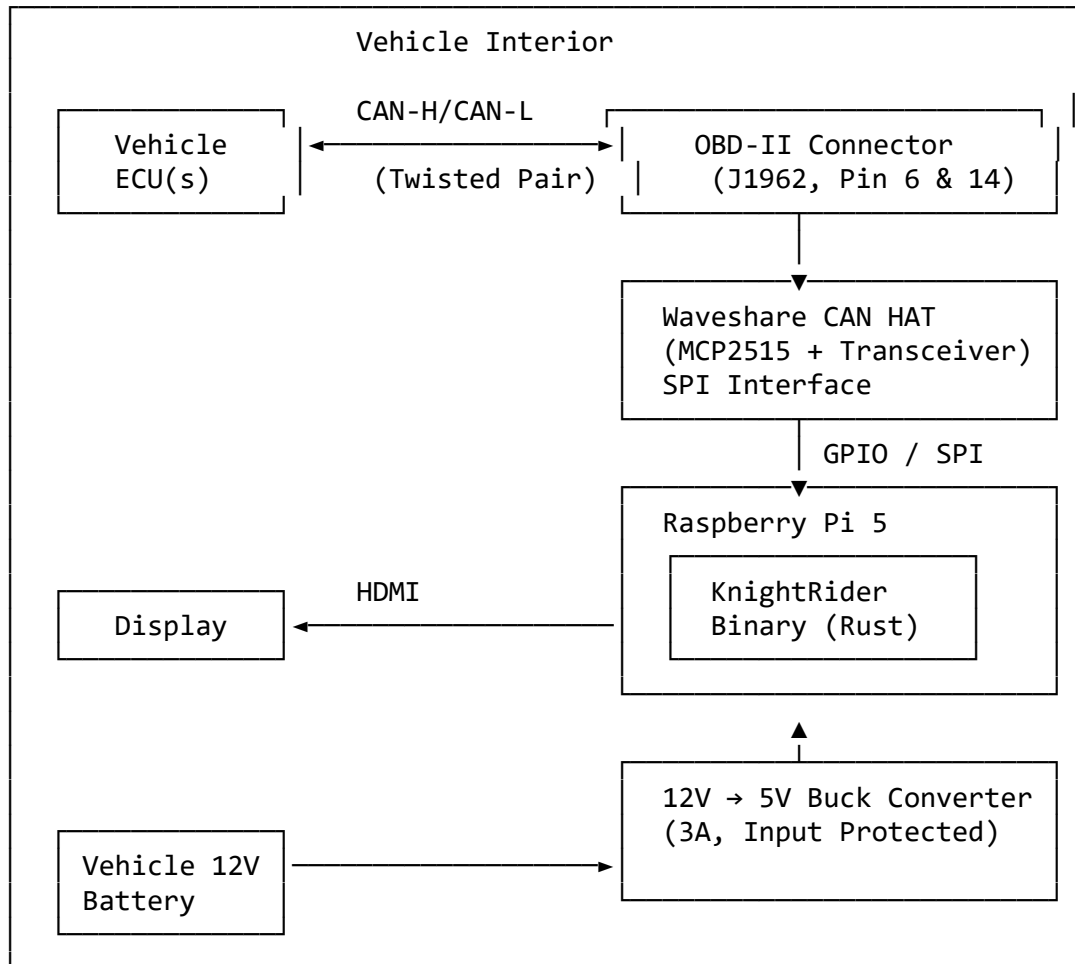
The hardware platform is composed of commodity components selected for availability, cost-effectiveness, and suitability for automotive deployment.

Component	Model / Specification	Purpose
Single-Board Computer	Raspberry Pi 5	Central processing unit. Runs the KnightRider diagnostic binary. ARM64 (aarch64) architecture.
CAN Interface	Waveshare 2-CH CAN HAT (MCP2515)	SPI-attached CAN controller. Provides physical CAN bus connectivity via the Pi's GPIO header.
OBD-II Connector	Standard 16-pin J1962 Male Connector	Vehicle interface. Pins 6 (CAN-H) and 14 (CAN-L) carry the CAN bus signals.
Power Supply	12V → 5V DC-DC Buck Converter, 3A minimum	Converts vehicle battery voltage (nominally 12V, peaks to 14.4V during charging) to the 5V required by the Raspberry Pi.
Display	HDMI-compatible monitor	For console output during development. Touch-based GUI is planned for v2.
Storage	MicroSD Card (32GB+)	Boot media and

Component	Model / Specification	Purpose
		local data logging storage.

Safety Note: The Raspberry Pi must never be powered directly from the vehicle battery. A dedicated buck converter with input protection is mandatory to prevent voltage spikes from damaging the Pi.

1.3.2 3.3.2 Hardware Integration Diagram



1.3.3 3.3.3 CAN Bus Physical Layer

The CAN bus uses a differential signalling scheme on two wires:

- **CAN-H (High)** - Pin 6 of the OBD-II connector
- **CAN-L (Low)** - Pin 14 of the OBD-II connector

The MCP2515 CAN controller on the Waveshare HAT communicates with the Raspberry Pi via the SPI bus. The kernel loads the `mcp251x` driver module, which exposes the CAN interface as a standard Linux network device (e.g., `can0`).

1.3.4 3.3.4 Power Management

The power subsystem must address the following challenges of the automotive environment:

1. **Voltage regulation** - Vehicle battery voltage ranges from 11.5V (cranking) to 14.4V (alternator charging). The buck converter must tolerate this range.
2. **Transient protection** - Load dumps and inductive spikes on the vehicle bus can exceed 40V. Input clamping or TVS diodes are recommended.
3. **Shutdown procedure** - The operating system requires a clean shutdown to prevent SD card corruption. An orderly shutdown -h now must precede power removal.

1.3.5 3.3.5 Physical Installation Considerations

Factor	Consideration
Mounting	The Pi assembly should be mounted in a vibration-dampened enclosure, away from heat sources (engine block, exhaust).
Cable Routing	The OBD-II cable should be routed away from high-voltage ignition leads to minimise electromagnetic interference.
Temperature	The Raspberry Pi 5 operates reliably between 0°C and 50°C. In-cabin placement is preferred over engine-bay mounting.
Humidity	The enclosure should provide IP52 protection or better to guard against condensation.

1.4 3.4 Software Architecture

1.4.1 3.4.1 Design Philosophy

The KnightRider software is built on the following architectural principles:

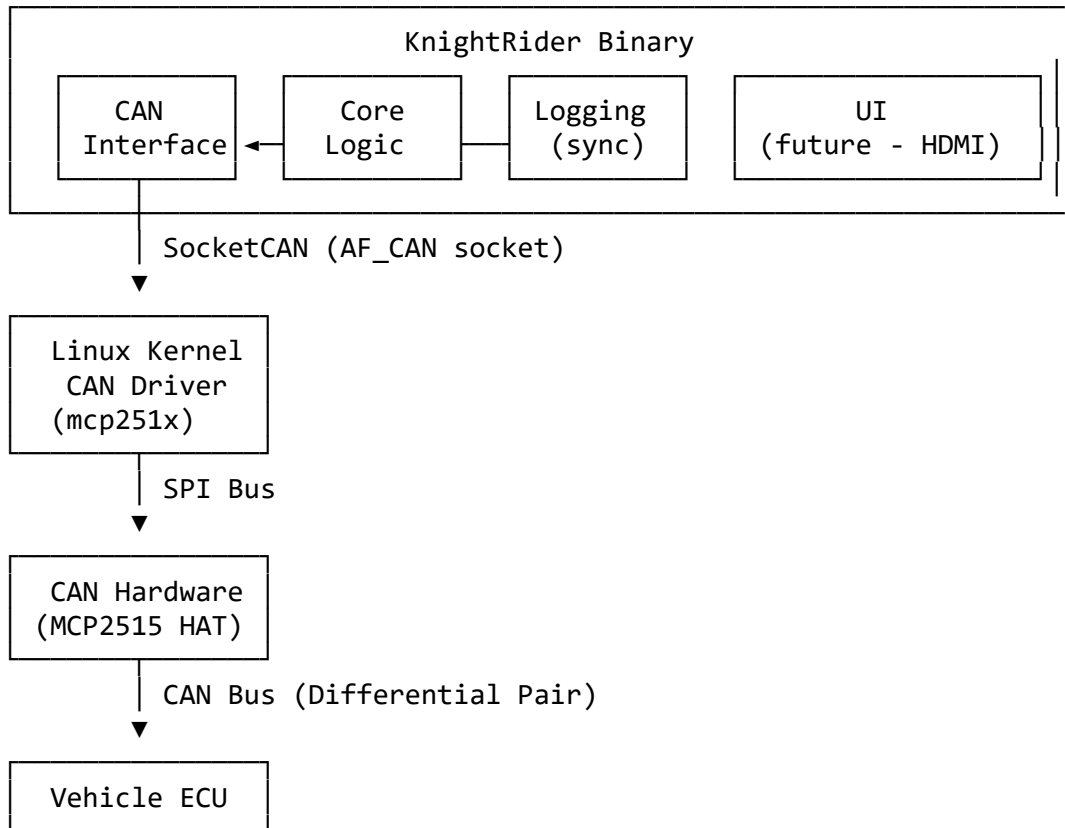
1. **Single binary** - The entire diagnostic tool compiles to one statically-linked executable with no runtime dependencies beyond the Linux kernel.
2. **No panics in production** - All functions return `Result<T, E>`. Errors are logged and handled gracefully.
3. **Minimal resource footprint** - Total runtime memory usage is under 2MB, well within the Pi 5's 8GB capacity.
4. **Single-threaded v1** - All operations execute in a single main loop using blocking I/O with timeouts. This eliminates synchronisation complexity while meeting the 100ms polling rate requirement.

1.4.2 3.4.2 Language Choice: Rust

Rust was selected as the implementation language for the following reasons:

Criterion	Justification
Memory safety	Rust's ownership model eliminates classes of bugs (use-after-free, buffer overflows, data races) that are particularly dangerous in automotive embedded systems.
Zero-cost abstractions	High-level constructs compile to efficient machine code with no runtime overhead.
No garbage collector	Deterministic memory management avoids unpredictable latency spikes during data acquisition.
Cross-compilation	Rust toolchain natively supports aarch64-unknown-linux-gnu target for ARM64 Raspberry Pi deployment.
Error handling	The <code>Result<T, E></code> pattern enforces explicit error handling at compile time, preventing silent failures.

1.4.3 3.4.3 Software Stack



1.4.4 3.4.4 Module Architecture

The codebase is organised into four top-level modules, each with clearly defined responsibilities:

1.4.4.1 Module 1: *can/* - CAN Bus Interface Layer

This module encapsulates all CAN bus communication, providing a clean abstraction over the Linux SocketCAN subsystem.

Sub-module	File	Responsibility
Interface	<code>interface.rs</code> (388 lines)	Raw SocketCAN socket operations: open, send, recv. Wraps AF_CAN sockets with timeout configuration via <code>setsockopt</code> . Provides platform-conditional compilation with full SocketCAN implementation on Linux, and a stub implementation on other platforms for cross-compilation.
ISO-TP	<code>isotp.rs</code> (513 lines)	ISO 15765-2 multi-frame message handling. Implements state machine for reassembling segmented CAN messages: Single Frames, First Frames, Consecutive Frames, and Flow Control. Maximum message size: 256 bytes.
OBD-II	<code>obd.rs</code> (241 lines)	OBD-II protocol layer (ISO 15031 / SAE J1979). Request building, response parsing, PID decoding with standard formulas, supported PID bitmask parsing, and negative response handling.
Scheduler	<code>scheduler.rs</code> (108 lines)	Request timing and rate limiting. Enforces 100ms minimum interval between requests (per ECU timing constraints) and 200ms response timeout.

1.4.4.2 Module 2: *core/* - Application Logic

Sub-module	File	Responsibility
State Machine	<code>state_machine.rs</code> (108 lines)	Manages application lifecycle through five states: IDLE → INITIALIZING → CONNECTED → RUNNING → ERROR. Implements automatic error escalation (5 consecutive errors trigger ERROR state) and recovery.

Sub-module	File	Responsibility
Signals	signals.rs (77 lines)	Decoded sensor values with ISO 8601 timestamps. Signal types: Engine RPM, Vehicle Speed, Coolant Temperature, Intake Air Temperature, Throttle Position, Fuel Tank Level, Timeout.
Datastore	datastore.rs (49 lines)	HashMap-backed in-memory storage for the latest signal values. Provides O(1) lookup by signal kind.

1.4.4.3 Module 3: Logging/ - Data Logging

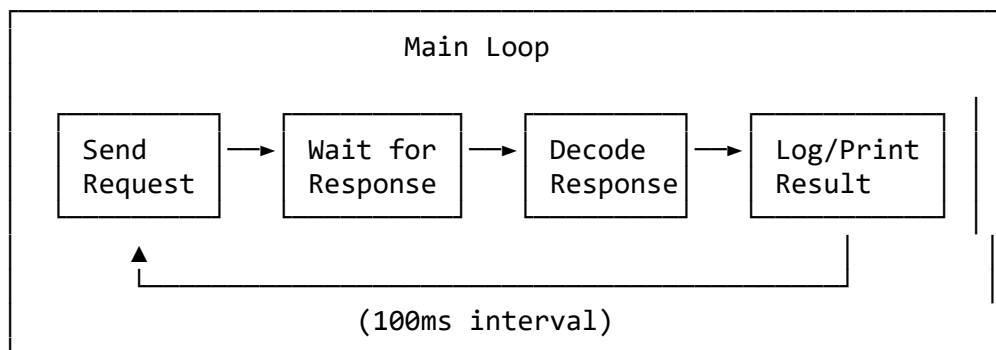
Sub-module	File	Responsibility
Ring Buffer	ringbuffer.rs (80 lines)	Generic fixed-size ring buffer using VecDeque. Default capacity: 10,000 entries (~30 seconds at 100 Hz polling). Oldest entries are overwritten when full.
Timeseries	timeseries.rs (113 lines)	CSV-format file writer for raw CAN frames. Logs timestamps (ISO 8601), CAN IDs, DLC, and raw data. Uses BufWriter with periodic flush (every 100 entries) to minimise SD card write cycles.

1.4.4.4 Module 4: ui/ - User Interface (Reserved)

Currently contains stub files (gauges.rs, renderer.rs, screens.rs) reserved for HDMI-based graphical UI development in v2. Out of scope for the current implementation.

1.4.5 3.4.5 Threading Model

KnightRider v1 uses a **single-threaded** architecture:



Design rationale: - No async runtime (tokio/async-std), which avoids unnecessary complexity and binary size inflation. - No threading primitives, which eliminates synchronisation overhead and data race potential. - Blocking I/O with timeouts, which is fully sufficient for the 10 Hz polling rate.

1.4.6 3.4.6 Memory Layout

Stack - CAN frame buffers (small) - Local variables
Heap - Ring buffer: ~1MB - ISO-TP reassembly: 4KB per session - Datastore: ~1KB (latest values)
Static / BSS - Logger configuration - PID definition tables

Total runtime memory: < 2MB

1.4.7 3.4.7 Build Configuration

The Rust project is configured for two build profiles:

Release Profile (for deployment on Raspberry Pi):

```
[profile.release]
opt-level = 3           # Maximum optimisation
lto = true              # Link-time optimisation for smaller binary
codegen-units = 1       # Single codegen unit for better optimisation
strip = true            # Remove debug symbols
panic = "abort"         # Abort on panic (no unwinding overhead)
```

Development Profile (for testing and debugging):

```
[profile.dev]
opt-level = 0           # No optimisation (fast compilation)
debug = true            # Full debug information
```

Cross-compilation for ARM64 deployment:

```
rustup target add aarch64-unknown-linux-gnu
cargo build --release --target aarch64-unknown-linux-gnu
```

1.4.8 3.4.8 Dependencies

The project uses a minimal dependency set, with platform-conditional compilation for Linux-specific libraries:

Crate	Version	Purpose
log	0.4	Structured logging facade

Crate	Version	Purpose
env_logger	0.11	Console log output with RUST_LOG filter
chrono	0.4	ISO 8601 timestamp formatting
serde / serde_json	1.0	Serialisation for configuration and test fixtures
socketcan	3.3	Linux SocketCAN bindings (<i>Linux only</i>)
libc	0.2	Low-level system calls for socket options (<i>Linux only</i>)
nix	0.29	POSIX API bindings for networking and polling (<i>Linux only</i>)

1.5 3.5 Data Acquisition Layer

1.5.1 3.5.1 OBD-II Protocol Overview

KnightRider communicates with vehicle ECUs using the OBD-II diagnostic protocol stack:

Standard	Layer	Description
ISO 15765-4	Physical + Data Link	CAN-based OBD-II. Defines electrical signalling, bit timing, and CAN frame format.
ISO 15765-2	Transport	ISO-TP. Provides segmentation and reassembly for messages exceeding the 8-byte CAN payload limit.
ISO 15031 / SAE J1979	Application	OBD-II diagnostic services. Defines modes, PIDs, and response formats.

1.5.2 3.5.2 CAN Addressing Scheme

OBD-II on CAN uses the following addressing convention:

Request Addresses:

CAN ID	Type	Description
0x7DF	Functional broadcast	Queries all ECUs simultaneously. Used for standard OBD-II requests.
0x7E0 to 0x7E7	Physical addressing	Targets a specific ECU. Rarely used in standard diagnostics.

Response Addresses:

CAN ID	Responding ECU
0x7E8	Engine Control Module (ECM), primary for powertrain data
0x7E9	Transmission Control Module (TCM)
0x7EA	Hybrid/Emissions module (varies by manufacturer)
0x7EB to 0x7EF	Manufacturer-specific ECUs

KnightRider broadcasts on 0x7DF and listens for responses in the 0x7E8 to 0x7EF range:

```
pub fn is_obd_response(can_id: u32) -> bool {  
    can_id >= 0x7E8 && can_id <= 0x7EF  
}
```

1.5.3 3.5.3 OBD-II Frame Format

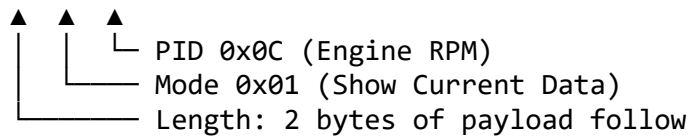
Request frame structure:

Byte:	0	1	2	3	4	5	6	7
	Len	Mode	PID	00	00	00	00	00

Example: Request Engine RPM (Mode 01, PID 0C):

CAN ID: 0x7DF

Data: 02 01 0C 00 00 00 00 00



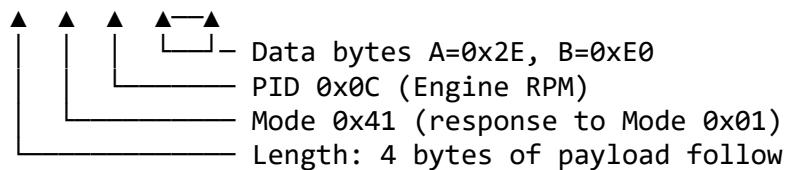
Response frame structure:

Byte:	0	1	2	3	4	5	6	7
	Len	Mode	PID	A	B	C	D	...

Example: Engine RPM Response (3000 RPM):

CAN ID: 0x7E8

Data: 04 41 0C 2E E0 00 00 00



$$\text{RPM} = ((A \times 256) + B) / 4 = ((0x2E \times 256) + 0xE0) / 4 = 3000$$

1.5.4 3.5.4 ISO-TP Transport Layer

For responses exceeding 7 bytes (the single-frame limit), KnightRider implements ISO 15765-2 (ISO-TP) message reassembly:

Frame Type	First Nibble	Description	Max Payload
Single Frame (SF)	0x0	Complete message in one CAN frame	7 bytes
First Frame (FF)	0x1	First fragment of multi-frame message	6 bytes
Consecutive Frame (CF)	0x2	Subsequent fragments	7 bytes
Flow Control (FC)	0x3	Receiver → Sender pacing	N/A

The IsoTpSession state machine handles reassembly:

1. **Single Frame** - Returns immediately with complete payload.
2. **First Frame** - Extracts total message length (12-bit), stores first 6 data bytes, sets `receiving = true`.
3. **Consecutive Frames** - Validates sequence numbers (0 to 15, wrapping), appends data until `expected_length` is reached.
4. **Error handling** - Reports sequence errors and overflow conditions.

Maximum supported message size: 256 bytes (more than sufficient for OBD-II Mode 01 responses).

1.5.5 3.5.5 Parameter Selection (PIDs)

KnightRider v1 captures the following OBD-II Mode 01 parameters:

PID	Parameter	Bytes	Formula	Unit	Range	Sampling Rate
0x00	Supported PIDs [01-20]	4	Bitmask	-	-	Once (init)
0x05	Engine Coolant Temperature	1	$A - 40$	°C	-40 to 215	1 Hz
0x0C	Engine RPM	2	$((A \times 256) + B) / 4$	rpm	0 to 16,383.75	10 Hz
0x0D	Vehicle Speed	1	A	km/h	0 to 255	10 Hz
0x0F	Intake Air	1	$A - 40$	°C	-40 to	1 Hz

PID	Parameter	Bytes	Formula	Unit	Range	Sampling Rate
	Temperature				215	
0x11	Throttle Position	1	$(A \times 100) / 255$	%	0 to 100	10 Hz
0x20	Supported PIDs [21-40]	4	Bitmask	-	-	Once (init)
0x2F	Fuel Tank Level	1	$(A \times 100) / 255$	%	0 to 100	0.1 Hz

Rationale for PID selection:

- **RPM, Speed, Throttle** - Primary powertrain health indicators. High sampling rate (10 Hz) enables detection of transient anomalies.
- **Coolant Temperature** - Critical engine health metric. Used for overheating detection.
- **Intake Air Temperature** - Ambient condition sensing. Influences fuel mixture calculations.
- **Fuel Tank Level** - Fleet management utility. Low sampling rate as this changes slowly.
- **Supported PIDs** - Queried once at initialisation to discover ECU capabilities.

1.5.6 3.5.6 Discovery Protocol

Before entering the main polling loop, KnightRider performs ECU discovery:

1. Constructs a Mode 01, PID 0x00 request (Supported PIDs [01-20]).
2. Broadcasts on 0x7DF.
3. Waits up to 500ms for responses from ECUs in the 0x7E8 to 0x7EF range.
4. Parses the 4-byte bitmask to determine which PIDs each ECU supports.
5. Logs discovered capabilities.
6. Transitions the state machine from INITIALIZING to CONNECTED.

If no ECU responds, the system logs a warning (“ECU may be offline”) but continues, allowing the operator to verify physical connections.

1.6 3.6 Data Processing Pipeline

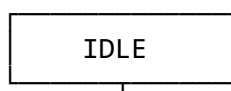
1.6.1 3.6.1 Main Loop Flow

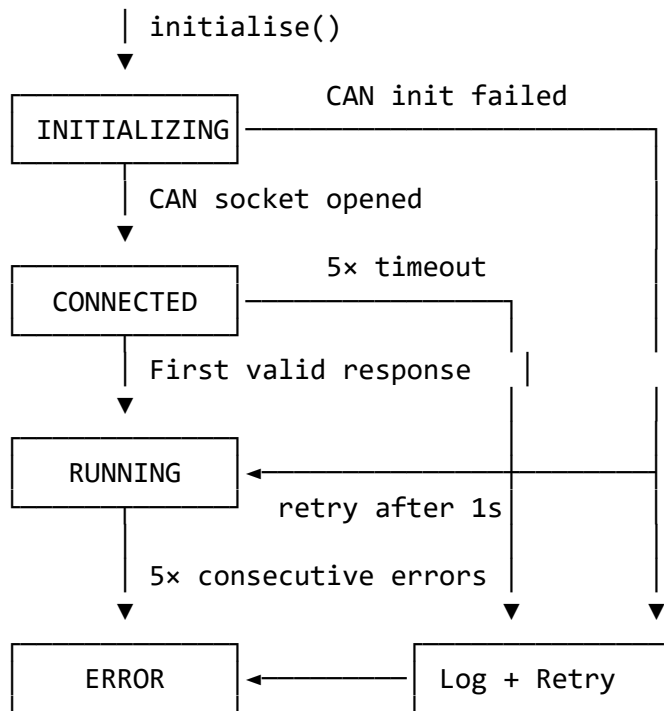
The central data acquisition loop executes the following sequence at a configurable interval (default: 100ms):

1. SCHEDULER: Wait until minimum request interval (100ms) has elapsed.
2. REQUEST: Build OBD-II request for target PID.
 - ↳ `ObdRequest::current_data(ObdPid::EngineRpm)`
 - ↳ `IsoTpSession::build_single_frame(&[0x01, 0x0C])`
 - ↳ `CanFrame::new(0x7DF, &[0x02, 0x01, 0x0C, 0x00, ...])`
3. TRANSMIT: Send CAN frame via SocketCAN.
 - ↳ `can.send(&frame)`
 - ↳ On failure: log warning, increment error counter, continue.
4. RECEIVE: Block on CAN socket with 200ms timeout.
 - ↳ `can.recv()`
 - ↳ Filter: Accept only 0x7E8 to 0x7EF (OBD-II response range).
 - ↳ On timeout: Log TIMEOUT signal, continue.
5. LOG RAW: Before any processing, write raw frame to CSV logger.
 - ↳ `RawFrameEntry { timestamp, can_id, dlc, data }`
 - ↳ `logger.log_frame(&entry)`
6. REASSEMBLE: Pass through ISO-TP session.
 - ↳ `isotp.receive(frame.data())`
 - ↳ Returns `Some(payload)` for single-frame, or `None` if multi-frame in progress.
7. PARSE: Extract OBD-II response fields.
 - ↳ `ObdResponse::parse(ecu_id, &payload)`
 - ↳ Validate mode (0x41 for Mode 01 response) and PID.
8. DECODE: Apply PID-specific formula.
 - ↳ `response.decode(ObdPid::EngineRpm)`
 - ↳ Returns `DecodedValue { value: 3000.0, unit: "rpm" }`
9. OUTPUT: Create Signal, print to console, update datastore.
 - ↳ `Signal::new(SignalKind::EngineRpm, 3000.0, "rpm")`
 - ↳ `println!("{}", signal.format_console())`
 - ↳ `[2025-12-31T14:23:45.123Z] RPM: 3000 rpm`
10. STATE: Update state machine.
 - ↳ On success: `record_success()` → transition to RUNNING.
 - ↳ On error: `record_error()` → after 5 errors, transition to ERROR.

1.6.2 3.6.2 Error Recovery

The state machine implements automatic error recovery:





Recovery behaviour: - After entering ERROR state, the system pauses for 1 second, then transitions back to CONNECTED. - The error counter resets on any state transition away from ERROR. - The system can recover from physical disconnection/reconnection without restart.

1.6.3 3.6.3 Data Logging Strategy

Raw CAN frames are logged to CSV before any processing occurs, ensuring forensic completeness:

Log format: (/tmp/knight-rider-raw.log)

```

timestamp,can_id,dlc,data
2025-12-31T14:23:45.123Z,0x7E8,5,04 41 0C 2E E0
2025-12-31T14:23:45.323Z,0x7E8,5,04 41 0C 2E E0

```

Write strategy: - BufWriter buffers writes in memory to minimise SD card I/O. - Flush occurs every 100 entries or at program termination. - The Drop implementation on TimeseriesLogger ensures a final flush on shutdown.

1.6.4 3.6.4 Timing Constraints

Parameter	Value	Description
Request interval	100ms	Minimum gap between OBD-II requests
Response timeout	200ms	Maximum wait for ECU response
Error threshold	5	Consecutive errors before entering

Parameter	Value	Description
		ERROR state
Recovery delay	1s	Pause before attempting recovery from ERROR
Log flush interval	100 entries	Buffered writes to reduce SD card wear

1.7 3.7 Testing & Validation Strategy

1.7.1 3.7.1 Virtual CAN Testing

The primary testing mechanism uses Linux's Virtual CAN (vcan) subsystem, which provides a software CAN bus without physical hardware:

Setup procedure:

Load the vcan kernel module

```
sudo modprobe vcan
```

Create virtual CAN interface

```
sudo ip link add dev vcan0 type vcan
```

Bring up the interface

```
sudo ip link set vcan0 up
```

A setup script (scripts/setup-vcan.sh) automates this process with error checking and idempotent operation.

Testing workflow:

Terminal 1: Run KnightRider

```
./target/release/knight-rider --interface vcan0
```

Terminal 2: Simulate ECU response (3000 RPM)

RPM formula: $RPM = ((A \times 256) + B) / 4$

$3000 \times 4 = 12000 = 0x2EE0$

```
cansend vcan0 7E8#04410C2EE0000000
```

1.7.2 3.7.2 Unit Tests

Each module contains embedded unit tests verifiable via `cargo test`. Key test coverage includes:

Module	Tests	Validates
can/interface.rs	4 tests	Frame construction, extended ID detection, hex formatting, data

Module	Tests	Validates
can/isotp.rs	8 tests	length validation Frame type detection, SF/FF/CF parsing, multi-frame reassembly, sequence error detection, single frame building
can/obd.rs	2 tests	RPM decoding formula accuracy, OBD response ID range validation
can/scheduler.rs	2 tests	Initial scheduling state, post-send timing behaviour
core/state_machine.rs	1 test	State transition sequence (IDLE to INIT to CONNECTED to RUNNING)
core/signals.rs	1 test	Console output format verification
core/datastore.rs	1 test	Store/retrieve cycle validation
logging/timeseries.rs	1 test	CSV format and timestamp accuracy
logging/ringbuffer.rs	1 test	Capacity enforcement and eviction behaviour

1.7.3 3.7.3 Integration Testing

Integration tests (in the tests/ directory) validate end-to-end data flow: - CAN frame construction, then ISO-TP framing, then OBD-II request building, then transmission. - Raw CAN response, then ISO-TP reassembly, then OBD-II parsing, then PID decoding, then signal output.

1.7.4 3.7.4 Real Vehicle Testing (Planned)

Phase 4 testing will be conducted at a partner workshop (Toyo-Tech) with the following protocol:

1. **Test vehicles** - Multiple makes (Toyota, Honda, Ford) to validate cross-brand compatibility.
2. **Baseline comparison** - Readings will be cross-referenced against commercial OBD-II tools to validate accuracy.
3. **Edge cases** - Testing during cranking (voltage dips), at idle, under load, and with intentional connector disconnection.
4. **Data recording** - All raw CAN traffic during testing sessions will be logged for later analysis and regression testing.

1.7.5 3.7.5 Validation Metrics

Metric	Target
RPM accuracy	±1 RPM compared to commercial scan tool

Metric	Target
Temperature accuracy	±1°C
Response rate	>95% successful responses at 10 Hz polling
Recovery time	<5 seconds from connector disconnection to resumed data flow
Uptime	Continuous operation for 8+ hours without crash or memory leak

1.8 3.8 Platform Portability

1.8.1 3.8.1 Cross-Platform Compilation Strategy

The codebase employs conditional compilation to ensure it compiles on both the target platform (Linux/ARM64) and the development platform (Windows/macOS):

```
// Linux: Full SocketCAN implementation
#[cfg(target_os = "linux")]
mod linux_impl { ... }

// Non-Linux: Stub implementation (compiles but returns NotSupported)
#[cfg(not(target_os = "linux"))]
mod stub_impl { ... }
```

This allows developer workflow (editing, compiling, running unit tests) to proceed on Windows while the final binary is cross-compiled for the Raspberry Pi's aarch64-unknown-linux-gnu target.

1.8.2 3.8.2 Deployment Procedure

1. Cross-compile on development machine: `cargo build --release --target aarch64-unknown-linux-gnu`
2. Transfer binary to Raspberry Pi via SCP.
3. Set up CAN interface on Pi: `sudo ./scripts/setup-vcan.sh` (for testing) or configure `can0` with hardware.
4. Execute: `./knight-rider --interface can0`

1.9 3.9 Limitations & Future Considerations

1.9.1 3.9.1 Current Limitations

Limitation	Impact	Mitigation Strategy
CAN-only protocol support	Pre-2008 vehicles using K-Line or J1850	CAN covers >85% of vehicles since 2008. Legacy protocol modules can be added in future.

Limitation	Impact	Mitigation Strategy
Mode 01 only	cannot be diagnosed. No DTC reading (Mode 03), freeze frame (Mode 02), or clearing (Mode 04).	These modes use the same CAN transport and can be added without architectural changes.
Console-only output	Requires HDMI monitor and keyboard for interaction.	Graphical UI module (ui/) is scaffolded for v2 development.
No persistent database	Session data is lost between runs (only raw CSV logs are retained).	SQLite or time-series database integration is planned.
Single-PID polling	v1 polls one PID per loop iteration.	The scheduler can be extended to cycle through a PID list per iteration.

1.9.2 3.9.2 Security Considerations

Concern	Current Status	Planned Mitigation
CAN bus write access	The system sends OBD-II requests (read-only diagnostics) on the CAN bus. No write-to-ECU operations are performed.	Mode 01 is inherently read-only. Future modes (e.g., DTC clearing) will require explicit user confirmation.
Physical security	The OBD-II port provides unrestricted access to the vehicle's CAN bus.	Physical access control is the vehicle owner's responsibility. KnightRider does not expose network endpoints in v1.
Data privacy	Diagnostic data remains local on the Pi (no cloud transmission).	All data stays on-device. Future cloud features (if added) will require explicit opt-in.

1.9.3 3.9.3 Planned Enhancements

Enhancement	Phase	Description
Multi-PID polling	v1.1	Cycle through all supported PIDs in round-robin fashion.

Enhancement	Phase	Description
DTC reading & clearing	v1.2	Implement OBD-II Modes 03 and 04 for diagnostic trouble code management.
Graphical UI	v2	HDMI-based touchscreen dashboard with real-time gauges, using the scaffolded ui/ module.
Mobile companion app	v2	Flutter-based mobile application communicating with the Pi over local WiFi.
Health scoring	v2	Composite vehicle health score derived from multiple sensor readings and trend analysis.
Brand-specific PIDs	v3	Configuration-driven PID mapping for manufacturer-specific extended diagnostics.
Predictive maintenance	v3	Machine learning models trained on historical sensor data to anticipate component failures.
Cloud data pipeline	v3	Optional data upload to cloud (FastAPI backend + S3) for fleet-wide analytics.

1.10 3.10 Conclusion

This chapter has presented the complete system architecture and methodology of KnightRider, from the physical OBD-II connector to the decoded sensor values printed to the operator's console. The architecture is designed around three core principles: **reliability** (Rust's memory safety, defensive error handling, automatic recovery), **simplicity** (single binary, single thread, minimal dependencies), and **extensibility** (modular design, reserved UI module, cross-platform compilation).

The evolutionary prototyping methodology ensures that each subsystem is validated against real-world conditions before the next layer is built upon it. The v1 implementation establishes a solid foundation: a working CAN bus interface, compliant OBD-II protocol stack, robust error handling, and comprehensive raw data logging. This foundation directly supports the planned expansions including graphical UI, mobile connectivity, multi-PID cycling, and eventually predictive analytics, without requiring architectural rework.

The system achieves its primary goal of making automotive diagnostics accessible: a sub-\$100 hardware kit running open-source software that can reveal what a vehicle's own

computer knows about its health, without requiring proprietary tools or cloud subscriptions.